Tech Challenge — Fase 4

Microsserviços e Saga Pattern na Plataforma de Gestão de Oficina Mecânica

Pós-graduação FIAP — Tech Challenge 15SOAT — Fase 4

Integrantes:

Integrante	RM
Rafael Xavier	RM371756
Silas Furini	RM374087

Vídeo de demonstração (até 15 min): https://share.descript.com/view/UGNJAf9yzG6. O roteiro está em `docs/roteiro-video-fase-4.md`.

Ambiente de demonstração: a gravação e o deploy de avaliação usam o ambiente AWS Academy. O Kind é usado apenas como validação local e no CI, antes do deploy. As evidências que dependem do AWS Academy estão reunidas na seção 10 (tabela "Evidências AWS Academy").

Repositórios (um por microsserviço) e artefatos finais promovidos em `main` (baseline de 11/11/2026):

Serviço	Repositório	`sourceSha` (commit que construiu a imagem)	CI em `main`	Digest da imagem (GHCR)
OS Service (orquestrador da saga)	https://github.com/rafaelxvr/Tech-challenge-15SOAT	`d9b605bd445a2442af09cad9cdbeee4f4542f4f9`	run 42953101412 `success`	`sha256:704d51aa1f75dc3bce6943768de42d1a8021c69c29fe3b324c09cf2896eb1aa3`
Billing Service (orçamento e pagamento)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-billing	`140dba5dd782f985ce2d6b5e07acb5aed57114b1`	run 42953106505 `success`	`sha256:bb97e1295bc5a33b65839d61f4338c84cdc7f47340aeb59cce0837b6a2368e94`
Execution Service (diagnóstico e reparo)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-execution	`fe2d8844c6172e3215603d9f97f5da64be22fc3e`	run 42953113662 `success`	`sha256:e3832274dc29b5e32174bdffeec4c2e50bd05d481107ec8770d964e16fb3af18`

Cada imagem é publicada no GHCR (`ghcr.io/rafaelxvr/oficina-mecanica`, `oficina-billing-service` e `oficina-execution-service`) com tag igual ao `sourceSha`. Neste baseline o `sourceSha` é o commit de merge da promoção `develop` → `main` (core #127, billing #29, execution #54), que incluem o trace por etapa e a anotação de `sagaId` (seção 6). O Sonar Quality Gate passou nos três serviços (seção 7). O manifesto `infra/aws-academy/release-manifest.json` e os overlays `aws` fixam esses SHAs e digests; o `deliveryRef` de cada serviço é `refs/tags/fase4-entrega-2026-11-12`, a tag da entrega.

Limite: a tabela prova SHA, digest e run de CI. Ela não prova, por si, deploy no AWS Academy; a execução de 11/11 está na seção 10. Essa execução usou a tag de entrega (commits de merge posteriores aos `sourceSha` da tabela); a coleta não registrou o digest de cada pod em execução, então a igualdade com os digests acima não é afirmada. A assinatura/atestado das imagens não foi verificada.

Sumário

- 1. Resumo executivo
- 2. Arquitetura dos microsserviços
- 3. Saga orquestrada
- 4. Divisão de responsabilidades e tecnologias
- 5. CI/CD, validação local e Kind
- 6. Observabilidade (New Relic)
- 7. Qualidade e testes (SonarCloud)
- 8. Pagamento com Mercado Pago (sandbox)
- 9. Matriz de rastreabilidade do enunciado
- 10. Evidências AWS Academy
- 11. Limitações e riscos conhecidos

1. Resumo executivo

O desenho da Fase 4 divide a aplicação da oficina em **três microsserviços independentes**, cada um com repositório, pipeline, infraestrutura de execução e banco próprios. Uma **saga orquestrada** coordena as transações que atravessam os três (abertura da OS, orçamento, aprovação, pagamento, reserva de peças e reparo) e registra compensações para falhas recuperáveis.

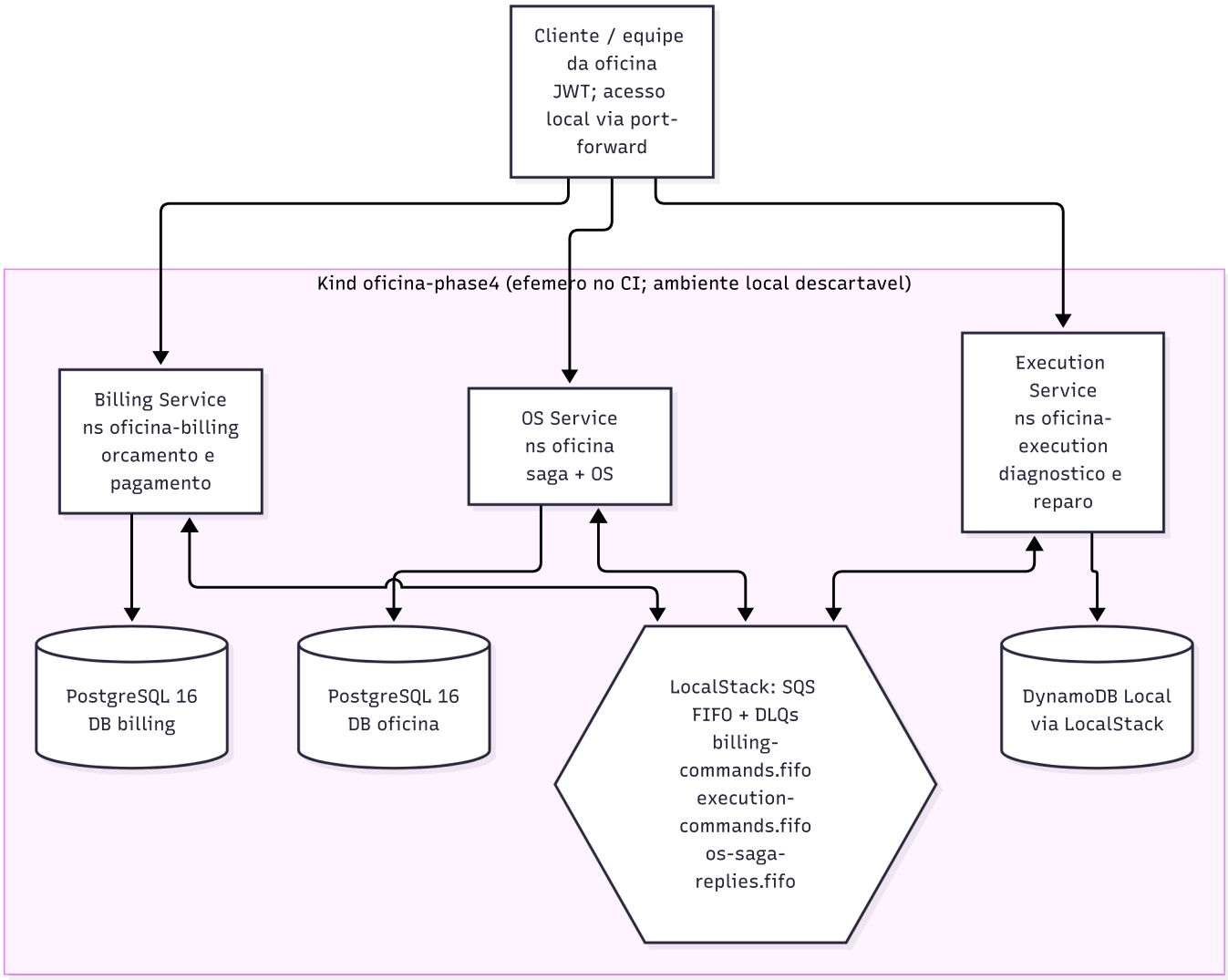
- **OS Service** é dono da ordem de serviço e orquestra a saga. Banco relacional PostgreSQL.
- **Billing Service** é dono do orçamento e do pagamento (Mercado Pago). Banco relacional PostgreSQL próprio.
- **Execution Service** é dono da fila de execução, do diagnóstico e do reparo. Banco não relacional DynamoDB.

Os serviços não acessam os bancos uns dos outros. A integração assíncrona usa filas SQS FIFO exercitadas localmente por LocalStack, com contratos JSON versionados. LocalStack é dependência de teste local; não representa provisionamento ou uso de uma conta AWS.

Este documento distingue "implementado e testado" (código, CI, Sonar, validação local no Kind) de "executado no ambiente AWS Academy". A execução no AWS Academy de 11/11/2026 está na seção 10, com seus resultados e limitações (estorno sandbox com falha e C3 só no Kind).

2. Arquitetura dos microsserviços

2.1 Diagrama geral



Fonte versionada do diagrama: `docs/architecture/phase4-overview.mmd` (SVG renderizado em `docs/architecture/rendered/phase4-overview.svg`).

Diagramas Mermaid versionados: `docs/architecture/c4-*.md` no repositório do OS Service e o diagrama do README do Billing Service. Diagrama geral da validação local Kind, reconciliado com os manifestos:

`docs/architecture/phase4-kind-geral.md` (não é evidência de deploy em nuvem).

2.2 Bancos isolados

Serviço	Banco no alvo local	Tipo	Justificativa
OS Service	PostgreSQL 16, banco `oficina`	SQL	OS, histórico e tabelas da saga precisam de atomicidade entre linhas
Billing Service	PostgreSQL 16 próprio, banco `billing`	SQL	Orçamento, pagamento, inbox e outbox precisam compartilhar transações; confere valor e moeda
Execution Service	DynamoDB Local via LocalStack	NoSQL	Fila e estado por OS usam acesso por chave e escritas condicionais atômicas

Nenhum serviço lê ou escreve no banco de outro. Dados de outro domínio só chegam por mensagem.

2.3 Mensageria e contratos

Fila local	Dona (consumidor)	Conteúdo
`billing-commands.fifo` e `billing-commands-dlq.fifo`	Billing	`GerarOrcamento`, `RegistrarDecisao`, `CancelarOrcamento`, `EstornarPagamento`
`execution-commands.fifo` e `execution-commands-dlq.fifo`	Execution	`IniciarDiagnostico`, `IniciarReparo`, `CancelarExecucao`
`os-saga-replies.fifo` e `os-saga-replies-dlq.fifo`	OS Service	respostas de Billing e Execution

- Envelope comum: `messageId`, `kind`, `type`, `schemaVersion`, `sagaId`, `osId`, `passo`, `causationId`, `occurredAt`, `correlationId`, `traceparent` e `payload`; tamanho máximo de 65.536 bytes.
- `MessageGroupId = osId` e `MessageDeduplicationId = messageId`. Cada resposta copia os identificadores de correlação do comando; `causationId` aponta para o `messageId` respondido.
- Os contratos canônicos são JSON Schemas `contracts/<tipo>/v1.json`: o consumidor é dono do schema de comando e o produtor é dono do schema de resposta. O OS Service fixa cópias de respostas para testes de contrato.
- Inbox idempotente e outbox são persistidos com a transação do caso de uso; as filas FIFO usam DLQ correspondente e `maxReceiveCount = 5`.
- No Kind, LocalStack atende em `http://localstack.infra.svc.cluster.local:4566` com serviços SQS e DynamoDB e credenciais fictícias `test/test`; não há conta nem credenciais reais de AWS.
- REST síncrono: cada serviço expõe sua própria API (`/billing`, `/execucao` e API do OS), com JWT e permissões validados nos próprios serviços, sem API Gateway ou ALB. Acesso local via `kubectl port-forward`. Especificações OpenAPI e coleção Postman: seção 9.

3. Saga orquestrada

3.1 Escolha e justificativa

Estratégia: orquestração, com o orquestrador no OS Service (`OsSaga`, tabela `os_saga`). Uma saga por OS começa na mesma transação que abre a OS. Decisão registrada na ADR 006 (`docs/adrs/006-orchestrated-saga.md`).

- Um único lugar guarda estado, etapa atual, prazos e o log de compensação. A falha fica legível, testável e demonstrável (`GET /api/ordens-servico/{id}/saga`).
- O ciclo de vida da OS já vivia no OS Service; o orquestrador fica ao lado do agregado que ele conduz.
- Reaproveita o outbox e o SQS FIFO da Fase 3, sem broker novo.
- Coreografia foi rejeitada: a lógica de compensação se espalharia por três repositórios e ciclos de eventos são fáceis de introduzir. RabbitMQ/Amazon MQ foi rejeitado: infraestrutura nova, sem reaproveitar o outbox, custo extra.
- Custo aceito: o OS Service conhece a ordem das etapas (acoplamento do orquestrador).

3.2 Fluxo feliz

Estados do ciclo da OS: `RECEBIDA` → `EM_DIAGNOSTICO` → `AGUARDANDO_APROVACAO` → `AGUARDANDO_PAGAMENTO` → `EM_EXECUCAO` → `FINALIZADA` → `ENTREGUE`; a rejeição do orçamento pode retornar a `EM_DIAGNOSTICO` com nova versão do orçamento. `CANCELADA` é terminal para os caminhos de falha. O estado e a etapa da saga são persistidos separadamente da enumeração da OS.

#	Etapa	Comando → destino	Resposta / avanço
1	Diagnóstico	`IniciarDiagnostico` → Execution	`DiagnosticoConcluido`; o mecânico conclui pela API do serviço
2	Orçamento	`GerarOrcamento` → Billing	`OrcamentoGerado`
3	Decisão do cliente	endpoint existente do OS envia `RegistrarDecisao` → Billing	`OrcamentoAprovado` com `checkoutUrl` ou `OrcamentoRecusado`
4	Pagamento	Mercado Pago ↔ Billing	Billing reconsulta o pagamento verificado e responde `PagamentoConfirmado`
5	Reserva de peças	ação local no OS Service	baixa de estoque com controle otimista; falha por estoque insuficiente inicia compensação
6	Início do reparo	`IniciarReparo` → Execution	`ReparoIniciado` imediato; `ReparoRecusado` se não houver capacidade/filial válida. Este é o pivô da saga
—	Conclusão	Execution conclui pela API do serviço	`ExecucaoFinalizada` encerra a saga e marca a OS `FINALIZADA`

3.3 Compensações

A compensação roda em ordem inversa, a partir de uma pilha persistida. Se o início do reparo for recusado, as ações anteriores são compensadas. `ReparoIniciado` aceito é o pivô; depois dele, a conclusão do reparo não desfaz etapas anteriores.

Falha	Motivo registrado	O que é compensado
Diagnóstico recusado	`DIAGNOSTICO_RECUSADO`	cancelar execução (no-op se nada foi criado) e cancelar a OS
Orçamento falhou ou não foi gerado no prazo	`ORCAMENTO_FALHOU` / `ORCAMENTO_SEM_RESPOSTA`	cancelar orçamento; se ainda não existir, a resposta pode trazer `orcamentoId` nulo; cancelar a OS
Cliente recusou o orçamento	decisão `RECUSADO`	cancelar orçamento; retornar a OS ao diagnóstico e repetir com a próxima versão do orçamento
Decisão expirou	`DECISAO_EXPIRADA` / `DECISAO_SEM_RESPOSTA`	cancelar orçamento e a OS
Pagamento recusado ou expirado	`PAGAMENTO_RECUSADO` / `PAGAMENTO_EXPIRADO`	cancelar orçamento e a OS
Estoque insuficiente na reserva	`ESTOQUE_INSUFICIENTE`	estornar pagamento, cancelar orçamento e a OS
Execution recusou o reparo	`REPARO_RECUSADO` / `REPARO_SEM_RESPOSTA`	repor estoque, estornar pagamento, cancelar orçamento e a OS

Prazos padrão: diagnóstico 3 dias, resposta automática 5 minutos, decisão 7 dias, pagamento 30 minutos e resposta de compensação 5 minutos. A compensação permite até 5 envios; depois disso, `FALHA_COMPENSACAO` exige tratamento pelo runbook.

3.4 Garantias de consistência

- Efeitos (comandos e ações locais), linha da saga e log de passos são gravados na **mesma transação**; o outbox publica depois do commit.
- Respostas são processadas uma vez (`saga_inbox`) e só aceitas quando `causationId` é igual ao comando aguardado; resposta atrasada ou duplicada não tem efeito.
- Toda espera tem prazo; um poller transforma espera vencida em falha e reenvia compensações não confirmadas, até 5 envios no total, antes de `FALHA_COMPENSACAO` (alerta, resolvida por runbook).
- Um pagamento confirmado **depois** de o orçamento ser cancelado é estornado automaticamente pelo Billing, sem resposta tardia para uma saga encerrada.
- `SAGA_ENABLED` e `SAGA_MENSAGERIA_ENABLED` ficam `false` por padrão. Só o perfil BDD/Kind os ativa depois que bancos e filas estiverem saudáveis; com a saga desativada, permanece o fluxo pass-through da Fase 3 e `GET /api/ordens-servico/{id}/saga` responde 404 antes de acessar o repositório.
- O acesso direto no Kind depende da autenticação JWT e das permissões validadas por cada serviço; não há rota de Lambda/API Gateway na validação local.
- OS criadas antes da saga (sem linha em `os_saga`) seguem o fluxo manual da Fase 3.

3.5 Evidência da saga

Evidência	Situação
Fluxo completo e compensações testados com BDD (Kind local)	Executado: run 40884415348 — 34 PASS, 2 cenários cobertos por teste unitário, 0 FAIL, cleanup zerado, incluindo caminho feliz, estoque insuficiente e reparo recusado. **Limite:** este run usou SHAs anteriores aos finais (core `7a8ea6c`, billing `17a37b9`, execution `1bf1e6c`). Posterior, no PR #126 (head `54a4e648e5b0016ead78797f4230cf210a8a53fd`, antes da promoção): run 42934949615 `success`, com a matriz BDD e a prova de cleanup verdes; é run de PR, não de `main`. Reexecução nos digests finais no AWS Academy: seção 10
Smoke e deploy local no Kind	Runs de `main` do baseline de 11/11: core 42953101412, billing 42953106505 e execution 42953113662, todos `success`. Prova local do trace por etapa no Kind: seção 6

4. Divisão de responsabilidades e tecnologias

4.1 Por que esta divisão

Serviço	Responsabilidade	Por que separado
OS Service	Abertura, status, histórico e consulta da OS; orquestração da saga	Dono do agregado central e do estado da saga
Billing Service	Orçamento (geração, envio, decisão); pagamento e estorno no Mercado Pago	Integração externa sensível, com segredos e webhook próprios; ritmo de mudança diferente
Execution Service	Fila de execução, diagnóstico e reparo; avisa a conclusão ao OS Service	Carga orientada a fila e acesso por chave, que casa com NoSQL

A divisão segue os contextos delimitados (atendimento, cobrança, execução) citados no enunciado.

4.2 Tecnologias

Item	OS Service	Billing Service	Execution Service
Linguagem e framework	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.2.5
Banco no alvo local	PostgreSQL 16	PostgreSQL 16 próprio	DynamoDB Local via LocalStack
Mensageria no alvo local	SQS FIFO via LocalStack	SQS FIFO via LocalStack	SQS FIFO via LocalStack
Contexto HTTP	API do OS	`/billing`	`/execucao`
Saúde	Actuator	`/billing/actuator/health`	`/execucao/actuator/health`
Imagem e manifestos	Docker, Kubernetes e Kind efêmero	Docker, Kubernetes e Kind efêmero	Docker, Kubernetes e Kind efêmero

5. CI/CD, validação local e Kind

5.1 Pipelines

Cada repositório tem pipeline independente de build, testes, cobertura, qualidade e deploy de validação em um cluster Kind descartável. O fluxo de branches é `feature` → PR para `develop` → integração → `main` como código entregue.

Item	Situação
Pipeline independente de build e testes nos três repositórios	Em `main` (baseline de 11/11/2026), todos `success` e com Sonar Quality Gate aprovado: core run 42953101412 (SHA `d9b605b`), billing run 42953106505 (`140dba5`) e execution run 42953113662 (`fe2d884`). SHAs completos e digests na capa
Cobertura e qualidade no CI	Seção 7
Deploy automatizado de validação	Kind efêmero no CI dos três serviços; prova local do trace por etapa no Kind com os heads dos PRs #126/#28/#53 (seção 6). Validação no AWS Academy: seção 10
Imagens e manifesto de release	Imagens publicadas no GHCR e fixadas por digest em `infra/aws-academy/release-manifest.json` (core PR #109). O manifesto prova SHA, digest e run de CI; não prova deploy no AWS Academy
Deploy de produção do core	Desabilitado por contrato (`APP_PRODUCTION_DEPLOYMENT_ENABLED` não definida); o job é pulado. Nenhum deploy de produção ocorreu

Item	Situação
Correções entregues em `main`	Health de mail desligado no perfil `aws` do core (PR #107); Flyway com credencial migradora no perfil `aws` do billing (PR #17); logs JSON com a mensagem real e redação segura nos três serviços (core PR #106, billing PR #16, execution PR #41); estorno e erro do provedor no billing (PR #23); propagação do contexto de trace pelo SQS (core #121, billing #24, execution #49); trace por etapa com `sagaId`/`correlationId` (core #126, billing #28, execution #53). Presença no histórico de `main`; a validação dessas correções no AWS Academy está na seção 10
Segurança	Gitleaks, SAST e SCA no CI sem bloqueio crítico/alto nos três serviços

5.2 Validação local pré-deploy no Kind

A validação local pré-deploy usa o cluster Kind `oficina-phase4`, efêmero no CI e descartável no desenvolvimento. A topologia usa namespaces `infra`, `data`, `oficina`, `oficina-billing` e `oficina-execution`; PostgreSQL 16 em serviços separados para OS e Billing; LocalStack para SQS FIFO, DLQs e DynamoDB Local; e imagens carregadas localmente no cluster.

- Nomes de serviço: `oficina-app:8080`, `oficina-billing:8081` e `oficina-execution:8082`.
- Dependências: `postgres-os.data.svc.cluster.local:5432` (`oficina`), `postgres-billing.data.svc.cluster.local:5432` (`billing`) e `localstack.infra.svc.cluster.local:4566` (`sqs,dynamodb`).
- O gate de banco usa uma credencial de migração separada da credencial de runtime: migração limpa até V11 e fixture V8 atualizada até V11. A role de runtime recebe apenas o DML necessário (`SELECT/INSERT/UPDATE`).
- O CI cria o cluster descartável, carrega a imagem, aplica manifestos, valida prontidão/saúde e destrói o cluster.
- Acesso local usa `kubectl port-forward` e validação JWT/roles nos próprios serviços. Não há API Gateway, ALB ou serviço externo de entrada na validação local.
- LocalStack usa somente credenciais fictícias. **Não há provisionamento de recursos AWS, deploy em nuvem, Terraform cloud apply nem credenciais AWS nesta validação local.**

6. Observabilidade (New Relic)

Critério de trace adotado. A saga não tem um `trace.id` único para a OS inteira: cada etapa disparada por um **gatilho HTTP** (criação da OS, ações do mecânico, decisão do cliente e simulação/confirmação de pagamento) tem **um `trace.id` próprio**, que atravessa os serviços pelo SQS dentro da etapa. A **correlação da OS inteira** é feita por `sagaId`, anotado como atributo nas transações, com a consulta NRQL `FROM Transaction SELECT ... WHERE sagaId = '<id>'` (por exemplo `SELECT uniqueCount(traceId) ... FACET traceId`; o atributo na conta é `traceId`). Este documento não afirma trace único da OS inteira.

Instrumentação. A instrumentação New Relic e a propagação de `traceparent`/`correlationId` estão em `main`: core #88, #89, #94; billing #8, #11; execution #29, #33; depois a propagação pelo SQS (#121, #24, #49) e o trace por etapa com `sagaId` (#126, #28, #53).

Prova local no Kind (sem AWS). Com os heads exatos dos PRs (core `54a4e648e5b0016ead78797f4230cf210a8a53fd`, billing `d68ca0640debf761d5e1767d7f1d8cb7cb085f4c`, execution `d9e727b297357065a24d9a72a2f21d766d5b92fa`), agentes Java reais e New Relic real, o cenário BDD-01a passou e a consulta `WHERE sagaId = <id>` retornou exatamente **4 `traceId` distintos**, um por etapa; as 12 transações com `sagaId` tinham todas `correlationId`; os consumidores de fila tinham `parent.transportType=Queue` e o `traceId` da etapa de origem; os três serviços apareciam. O CI do PR #126 fechou verde (run 42934949615). **Limites:** a verificação usou um driver descartável que gerou quatro raízes W3C distintas, uma por etapa; o resultado vale para esses heads, não para os SHAs de `main`; a prova não é de trace único da OS inteira. A prova no AWS Academy está na seção 10. A chave de licença é fornecida fora de banda e nunca versionada.

Item exigido no vídeo	Situação
Rastreamento distribuído com `traceparent` e `correlationId`	Critério: um `trace.id` por etapa HTTP + correlação da OS por `sagaId` (NRQL). Kind local: 4 traces por `sagaId` provadas. AWS Academy (11/11): atendido nos dois fluxos executados, seção 10

Item exigido no vídeo	Situação
Sinais `saga_prazo_falhou`, outbox `BLOCKED`, idade do item mais antigo, profundidade de DLQ e falha de compensação	Evidência de execução não anexada; ver o runbook de operação
M4: 20 ou mais sagas expiradas distintas sem processar em varreduras consecutivas	Limiar e notificação não provados
M7: alerta antes/no limite de cinco envios de compensação ou quando envio bloqueado atingir 20 minutos	Limiar e notificação não provados; o runbook cobre redrive/reconciliação seguros
Dashboard de saga (volume, duração e falhas)	Não anexado

7. Qualidade e testes (SonarCloud)

SonarCloud está habilitado nos três repositórios (`SONAR_ENABLED=true`) por core #90 e #92, billing #9 e execution #30, #31.

Medição em `main` (baseline de 11/11/2026). O Sonar Quality Gate **passou nos três serviços** nos SHAs do baseline. A **cobertura em código novo** (`new_coverage`, mínimo 80%), obtida no check SonarCloud do SHA de `main`, é: core 91,1%, billing 92,6% e execution 93,4%. **São valores de cobertura em código novo, não de cobertura total; a cobertura total de cada repositório não foi medida.**

Serviço	SHA de `main`	Quality Gate	Cobertura em código novo (`new_coverage`, mín. 80%)	Run
OS (core)	`d9b605bd445a2442af09cad9cdbeee4f4542f4f9`	aprovado	91,1%	42953101412
Billing	`140dba5dd782f985ce2d6b5e07acb5aed57114b1`	aprovado	92,6%	42953106505
Execution	`fe2d8844c6172e3215603d9f97f5da64be22fc3e`	aprovado	93,4%	42953113662

Requisito	Situação
Build e testes independentes	Runs `success` nos três serviços em `main` (seção 5.1)
Fluxo completo e compensações com BDD	Kind local: run 40884415348 (SHAs anteriores aos finais) e run 42934949615 (PR #126). Reexecução no AWS Academy: seção 10
Cobertura mínima de 80% por serviço	Atendida em código novo (Quality Gate aprovado nos três `main`); cobertura total não medida
Quality Gate no CI	Aprovado nos três `main`

8. Pagamento com Mercado Pago (sandbox)

O Billing Service integra o Mercado Pago por uma camada anticorrupção (`GatewayPagamento`): preferência de checkout, consulta de pagamento, busca por `external_reference` e estorno. A notificação do Mercado Pago nunca é confiada: o pagamento é relido no provedor e `external_reference`, valor e moeda (BRL) são conferidos antes de confirmar; divergência não confirma e estorna.

- Webhook `POST /billing/webhooks/mercadopago` com assinatura HMAC, janela de tempo, deduplicação e limites de corpo e taxa.
- Sem túnel HTTPS, o webhook real não alcança o Kind; a alternativa suportada é a reconsulta (`POST /billing/pagamentos/{id}/reconsulta`), que usa o mesmo caminho de verificação.
- O gateway fake existe só no perfil de testes e **não** é evidência de pagamento real.
- Coleção e environments Postman: PR #85.
- Os segredos `MP_ACCESS_TOKEN` e `MP_WEBHOOK_SECRET` são injetados fora de banda e nunca versionados ou registrados em logs.
- Pagamento sandbox real confirmado no caminho feliz (R$ 130,00); estorno real **não** concluído (HTTP 401 no sandbox): seção 10.

8.1 Correção de preços do catálogo (centavos → reais)

- **Defeito:** o seed `V2__seed_data.sql` gravou preços em centavos (ex.: `12000`) em colunas `NUMERIC(10,2)` cujo contrato é reais decimais, de modo que `12000` valia R$ 12.000,00 em vez de R$ 120,00. Num checkout de teste, isso cobrou R$ 17.000 onde o esperado era R$ 170.
- **Correção:** `V11__corrigir_precos_semeados_para_reais.sql` (core PR #117) divide por 100 somente as 10 linhas de serviços e as 10 de peças do seed que ainda coincidem com a impressão digital completa gravada pelo V2; linha editada depois do seed não é tocada e a migration é idempotente. Ordens de serviço já criadas têm snapshots de preço próprios e não são alteradas. O contrato de release avançou para V1–V11 (ver `docs/runbooks/first-writer-cutover.md`).
- **Preços do catálogo após a V11 (R$):** serviços: Troca de Óleo 80,00; Alinhamento 120,00; Balanceamento 80,00; Revisão Geral 350,00; Troca de Pastilha 180,00; Troca de Pneu 50,00; Diagnóstico Eletrônico 80,00; Troca de Filtro de Ar 40,00; Troca de Correia Dentada 450,00; Higienização do A/C 150,00. Peças (valor unitário): Óleo Motor 5W30 1L 25,00; Filtro de Óleo Universal 15,00; Pastilha Freio Dianteira 80,00; Pastilha Freio Traseira 70,00; Filtro de Ar Universal 20,00; Correia Dentada + Kit 180,00; Vela de Ignição NGK 35,00; Líquido de Arrefecimento 1L 18,00; Filtro de Combustível 25,00; Amortecedor Dianteiro (par) 180,00.
- **Verificação:** em execução anterior no AWS Academy, com a V11 aplicada, o caminho feliz concluiu com cobrança de R$ 170,00 (saga `CONCLUIDA`, OS `FINALIZADA`). Essa execução usou um baseline de imagens anterior ao final; a execução de 11/11/2026 no AWS Academy (R$ 130,00, saga `CONCLUIDA`, OS `ENTREGUE`) está na seção 10.

9. Matriz de rastreabilidade do enunciado

Legenda: **Evidenciado** (run/PR e SHA citados); **Parcial** (evidência de SHA anterior ou só configuração); **AWS Academy** (depende da seção 10).

Requisito	Estado	Evidência verificável	Lacuna
Três serviços independentes, persistência isolada (SQL + NoSQL)	Evidenciado (Kind)	Repositórios, SHAs e digests na capa; seção 2.2; smokes de isolamento e grants negativos no run 40884415348	Revalidação nos digests finais no AWS Academy
APIs REST documentadas	Parcial	Core `docs/openapi.yaml`; billing `docs/openapi.json`; Springdoc nos três; coleção Postman #85; spec do execution em PR #45; coleção validada offline por `python scripts/postman/validate-collection.py`	Spec do execution depende do merge do PR #45
Mensageria, contratos, inbox/outbox, Saga	Evidenciado	Seções 2.3 e 3; run 40884415348: 34 PASS, 2 por teste unitário, 0 FAIL	—
Fluxo feliz ponta a ponta	Evidenciado (Kind e AWS Academy)	BDD-01a no mesmo run, cleanup zerado; caminho feliz no AWS Academy em 11/11/2026: saga `CONCLUIDA`, OS `ENTREGUE`, R$ 130,00 (seção 10); execução anterior com R$ 170,00 (seção 8.1)	—
Compensação e idempotência (EstoqueInsuficiente, ReparoRecusado, duplicata, resposta tardia, DLQ/redrive)	Evidenciado (Kind, SHAs anteriores)	Matriz BDD do run 40884415348; dois cenários cobertos por teste unitário (explícito)	No AWS Academy: estorno real falhou (HTTP 401, defeito aberto) e ReparoRecusado não foi executado (seção 10)
Mercado Pago sandbox real + estorno	Parcial: pagamento sim, estorno **não**	Pagamento real confirmado no caminho feliz do AWS Academy (R$ 130,00 em 11/11; R$ 170,00 em execução anterior); correção do estorno em billing #23, testada com doubles; Postman #85	Estorno real não concluído: HTTP 401 do provedor nas 3 tentativas, saga em `FALHA_COMPENSACAO` (seção 10)
New Relic e rastreamento distribuído	Evidenciado (Kind e AWS Academy)	Seção 6: Kind com 4 `traceId` por `sagaId`; AWS Academy em 11/11: caminho feliz 12 transações/5 traces e C2 11 transações/4 traces, 3 serviços (seção 10)	—
Cobertura ≥ 80% e SonarCloud	Atendido em código novo	Seção 7: Quality Gate aprovado nos três `main`; 91,1%, 92,6% e 93,4%	Cobertura total não medida

Requisito	Estado	Evidência verificável	Lacuna
CI/CD e política de PR	Evidenciado	Seção 5.1; PRs para `develop` e promoção para `main`; Gitleaks/SAST/SCA sem bloqueio crítico/alto	Deploy de produção do core desabilitado por contrato
Kind / Kubernetes automatizado	Evidenciado	Runs `success` nos três serviços em `main`; prova local do trace por etapa (seção 6)	O Kind é validação pré-deploy; o deploy de avaliação é no AWS Academy
Deploy de avaliação AWS Academy	Evidenciado (com limite)	Terraform e overlay (PR #95), manifesto de release (PR #109); execução de 11/11 na tag de entrega, destruída, inventário final zerado (seção 10)	Digest por pod não registrado; C2 e C3 incompletos (seção 10)
Runbook de DLQ/redrive e reconciliação	Parcial	Harness #91 e runbook #93	DLQs e filas em 0 e sem duplicidade no AWS Academy (seção 10); redrive não executado lá
Vídeo ≤ 15 min	Entregue	https://share.descript.com/view/UGNJAf9yzG6; roteiro em `docs/roteiro-video-fase-4.md`	—

10. Evidências AWS Academy

Execução de 11/11/2026 no AWS Academy (us-east-1), conduzida pelo Rafael no Postman (ambiente `aws-academy`), sobre a tag de entrega `fase4-entrega-2026-11-12`. Os resultados abaixo são os observados, inclusive o defeito aberto no estorno (C2) e o cenário não executado (C3); nenhum item é declarado como atendido além do que a evidência mostra. Os `sagaId` são sintéticos.

Evidência	Resultado	Referência (run, comando, captura)
Deploy no AWS Academy (health por serviço e teardown com inventário zerado)	**Atendido.** Os três serviços executaram os fluxos abaixo via SQS. Teardown: `terraform destroy` concluído na 2ª passada (a 1ª estourou 10 min), "Destroy complete! Resources: 5 destroyed". Inventário final zerado em us-east-1 (EKS, RDS, EC2, ELB, ENI, EBS e snapshots, SG e VPC não padrão, SQS, DynamoDB, NAT/EIP, launch templates, ECR); restam apenas log groups do próprio lab	Leitura somente leitura de ~20:29Z; Service LoadBalancer/Ingress ausentes antes do destroy
Caminho feliz (C1)	**Atendido.** Saga `CONCLUIDA` (19 passos, sem compensação), OS `ENTREGUE`, pagamento `CONFIRMADO` de R$ 130,00 (Mercado Pago sandbox, confirmado por reconsulta)	Postman, pasta 03; saga `791d535f-6b5b-4d74-96be-88b049889c98`
EstoqueInsuficiente com estorno real concluído no Mercado Pago sandbox (C2)	**Não atendido (defeito aberto).** Estoque insuficiente detectado e compensação disparada, mas o estorno falhou: `MERCADO_PAGO_ESTORNO` HTTP 401 nas 3 tentativas, enquanto busca e consulta com o mesmo token retornaram 200. Resultado: saga `FALHA_COMPENSACAO`, OS `AGUARDANDO_PAGAMENTO`, pagamento `CONFIRMADO` de R$ 160,00 sem estorno (esperado: `CANCELADA`/`ESTORNADO`). Causa não determinada (permissão/escopo do token sandbox para estorno ou limitação do sandbox); o billing repassa o 401	Postman, 05.5; saga `2d8b4a15-d872-439d-b4a1-f62ddeecbfe9`; log de provedor do billing, sanitizado. Mesmo defeito do billing #23, não fechado
ReparoRecusado e compensação (C3)	**Não executado no AWS Academy** (tempo da janela). Provado apenas no Kind: matriz BDD do run 40884415348, em SHAs anteriores aos finais	Seção 3.5
Trace por etapa e consulta NRQL por `sagaId`	**Atendido** nos dois fluxos executados. Caminho feliz: 12 transações, 5 traces (um `trace.id` por etapa), 3 serviços, entrada via fila SQS. C2: 11 transações, 4 traces, 3 serviços (sem etapa de reparo, pois a saga parou na compensação). Veredito do verificador: aprovado nos dois	NRQL: `FROM Transaction SELECT count(*) WHERE sagaId = '<id>' FACET traceId SINCE 3 hours ago`, conferido por script somente leitura (NerdGraph)
Filas, DLQs e idempotência	**Atendido.** `billing-commands`, `execution-commands`, `os-saga-replies` e `notifications`: 0 mensagens visíveis, em voo ou atrasadas; as 3 DLQs: 0/0/0. 1 pagamento por OS nas 4 OS da sessão; 0 passos de saga duplicados	Leitura de ~20:15Z. As outras 2 sagas (checkout expirado, `CANCELADA`) foram descartadas

Evidência	Resultado	Referência (run, comando, captura)
BDD, Sonar e New Relic nos digests finais no AWS Academy	**Parcial.** New Relic: atendido (linha do trace). BDD automatizado (`bdd.mjs`) não foi executado no AWS Academy; o caminho feliz foi gravado pelo Postman. Sonar não foi reexecutado: vale o Quality Gate dos três `main` (seção 7)	Seções 3.5 e 7
Tag de entrega `fase4-entrega-2026-11-12` criada nos três repositórios	**Atendido.** Tag existente nos três; commits de merge `develop` → `main`: core `97801723` (#131), billing `58954fe5` (#31), execution `50c688d9` (#56)	Verificado via API do GitHub
URL do vídeo de demonstração	**Entregue:** https://share.descript.com/view/UGNJAf9yzG6	Capa e roteiro

11. Limitações e riscos conhecidos

- **Estorno no Mercado Pago sandbox (defeito aberto):** no AWS Academy o estorno da compensação de EstoqueInsuficiente falhou com HTTP 401 nas 3 tentativas (busca e consulta com o mesmo token deram 200); a saga terminou em `FALHA_COMPENSACAO` com o pagamento `CONFIRMADO` sem estorno. Causa não determinada; pode ser escopo do token sandbox ou limitação do sandbox. A detecção do estoque insuficiente e o disparo da compensação foram provados; o estorno real, não.
- **C3 (ReparoRecusado):** não executado no AWS Academy por tempo de janela; provado apenas no Kind (run 40884415348, SHAs anteriores).
- **Deploy de produção:** desabilitado por contrato; nenhum deploy de produção ocorreu.
- **Webhook Mercado Pago:** sem túnel HTTPS, a confirmação usa reconsulta, não a notificação real.
- **Compensação:** `FALHA_COMPENSACAO` após cinco envios exige tratamento pelo runbook; não é garantia de recuperação automática.
- **Evidência em SHAs anteriores:** BDD e smokes principais do Kind foram executados em SHAs anteriores aos finais; a prova do trace por etapa usou os heads dos PRs #126/#28/#53, não os SHAs de `main`.
- **Trace:** o critério é um `trace.id` por etapa HTTP mais correlação por `sagaId`; não há trace único da OS inteira.
- **Cobertura:** os percentuais são de código novo; a cobertura total não foi medida.
- **Proteção de `main`:** na última leitura (03/11/2026), o core tinha proteção de branch sem checagens obrigatórias e billing e execution não tinham proteção.
- **Credenciais:** Mercado Pago e New Relic ficam fora de banda; nada neste documento contém segredo.